

## PAULT DOCUMENTATION

# 2026 02 21 Versioning V2 Design

Source: docs/plans/2026-02-21-versioning-v2-design.md

**Theme:** Clean Technical

Generated: February 26, 2026

# Prompt Versioning v2 — Design Document

**Date:** 2026-02-21

**Status:** Approved

## Context

Pault's Prompt Versioning (Pro feature) currently provides basic snapshots, a side-by-side text diff (no change highlighting), and one-click restore. Users cannot see what actually changed between versions, metadata like tags/variables is not versioned, there's no way to manage or search versions, and restore happens without preview. This overhaul addresses all four gaps to make versioning production-grade.

## Scope

Four improvement areas, implemented as one cohesive overhaul:

1. **Better Diffs** — line+character-level highlighting, inline & side-by-side modes
2. **Richer Snapshots** — version all mutable fields (content, title, tags, variables, favorite)
3. **Version Management** — delete, search/filter, version count badge, configurable pruning
4. **Restore Workflow** — preview with diff, partial restore (pick which fields), arbitrary version comparison

## Design Decisions

### Diff Engine: Pure Swift built-in `CollectionDifference`

- Two-pass: line-level first, character-level within changed lines
- Reuses pattern from existing `RefinementLoopView.swift` `DiffView`
- Zero external dependencies (matches project convention)
- New file: `DiffEngine.swift` (pure logic, no UI)

### Model: Hybrid explicit + JSON blob

- Keep existing: `title`, `content`, `savedAt`, `changeNote`
- Add explicit: `isFavorite: Bool` (queryable)
- Add blob: `snapshotData: Data?` — JSON-encoded `VersionSnapshot` (tags, variables)
- Automatic `SwiftData` migration (optional/defaulted fields)

### Dedup Guard

- `saveSnapshot()` checks if anything actually changed vs. latest version before creating a new snapshot

## Files to Modify

| File | Changes |

|-----|-----|

| Pault/Models/PromptVersion.swift | Add `isFavorite`, `snapshotData`, `VersionSnapshot`  
Codable struct |

| `Pault/Services/PromptService.swift` | Update `saveSnapshot()` to capture all fields, add dedup guard, configurable pruning limit |

| `Pault/Views/PromptDiffView.swift` | Overhaul with inline/side-by-side toggle, diff highlighting, metadata changes section, arbitrary version comparison, restore preview with partial restore |

| `Pault/Views/PromptVersionHistoryView.swift` | Add swipe-to-delete, multi-select delete, search/filter bar, compare-two-versions selection |

| `Pault/Views/InspectorView.swift` | Add version count badge to History tab |

| `Pault/Views/PreferencesView.swift` | Add configurable "Max versions per prompt" stepper |

## New Files

| File | Purpose |

|-----|-----|

| `Pault/Services/DiffEngine.swift` | Pure diff logic: line-level + character-level diffing |

## Existing Code to Reuse

| Code | Location | Reuse |

|-----|-----|-----|

| Word-level diff pattern | `Pault/RefinementLoopView.swift:18-44` | Generalize into `DiffEngine` |

| `createTag()` deduplication | `Pault/Services/PromptService.swift:99` | Use during tag restore |

| `TemplateEngine.syncVariables()` | Template variable parsing | Re-sync after content restore, then overlay defaults |

| `ProStateManager` checks | Throughout views | Gate new UI behind Pro |

## Implementation Phases

### Phase 1: Model + Snapshot Enrichment

1. Define `VersionSnapshot` Codable struct in `PromptVersion.swift`
2. Add `isFavorite: Bool` and `snapshotData: Data?` to `PromptVersion`
3. Add computed property to encode/decode `VersionSnapshot`
4. Update `saveSnapshot()` in `PromptService` to capture all fields + dedup guard
5. Wire configurable pruning limit via `@AppStorage("versionHistoryLimit")` in `PreferencesView`

### Phase 2: Diff Engine

1. Create `DiffEngine.swift` with `LineDiff/CharacterDiff` types
2. Two-pass algorithm: line-level then character-level refinement
3. Field comparison helpers for metadata

### Phase 3: Enhanced Diff UI

1. Refactor PromptDiffView to use DiffEngine
2. Segmented control: Inline | Side-by-Side
3. Collapsible metadata changes section
4. Second version picker for arbitrary comparison

### Phase 4: Version Management

1. Swipe-to-delete and multi-select delete
2. Search/filter by change note and date
3. Version count badge in Inspector
4. Configurable pruning limit in Preferences

### Phase 5: Restore Workflow

1. Preview sheet showing diff of what will change
2. Partial restore checkboxes (Content, Title, Tags, Variables, Favorite)
3. Enriched "before restore" snapshots

## Key Technical Details

### Tag Restore Sequence

1. Remove current tag associations
2. For each snapshot tag: find by name or create new
3. Associate with prompt

### Variable Restore Sequence

1. Set content from snapshot
2. `TemplateEngine.syncVariables()` to re-derive
3. Overlay snapshot defaults onto matching variables

### Dedup Guard

Compare content, title, isFavorite, tag names, variable names+defaults against latest version. Skip if unchanged.

### Arbitrary Version Comparison

"Compare" mode in version list — select two versions via checkboxes, open diff with both.